

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Database Management Systems

COURSE CODE: CSA05

COURSE OUTCOME COVERED (CO4): Design database solutions incorporating file organization, indexing, and hashing techniques to optimize data storage and retrieval efficiency. (BL3, BL4)

Activity Tool: Code Challenge (High) Assessment Tool Weightage: 35% Total Marks: 50

Code of Conduct

I P SHASHANK (192524282) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: P SHASHANK

Name: P SHASHANK **Reg. No:** 192524282

Question 1: Heap File Organization (Insert, Delete, Sequential Scan)

Bloom's Level: BL3 – Apply | Marks: 5

Question:

Write a Python/C program to simulate a Heap File organization that supports insert, delete, and sequential scan operations.

Program Code:

```
"""
Q1: Simulate a Heap File Organization
Supports: insert, delete, and sequential scan operations.
In a heap file, records are stored in the order they arrive (no sorting).
Delete is done by marking a record as deleted (tombstone) - a common
technique in real heap file implementations to avoid shifting all records.
"""

class HeapFile:
    def __init__(self):
        self.records = [] # list of dicts: {'id', 'data', 'deleted'}

    def insert(self, rec_id, data):
        record = {'id': rec_id, 'data': data, 'deleted': False}
        self.records.append(record)
        print(f"Inserted record -> ID:{rec_id}, Data:{data}")

    def delete(self, rec_id):
        for rec in self.records:
            if rec['id'] == rec_id and not rec['deleted']:
                rec['deleted'] = True
                print(f"Deleted record with ID:{rec_id}")
                return
        print(f"Record with ID:{rec_id} not found")

    def sequential_scan(self):
        print("\n--- Sequential Scan (Heap File Contents) ---")
        found_any = False
        for rec in self.records:
            if not rec['deleted']:
                print(f"ID:{rec['id']} Data:{rec['data']}")
                found_any = True
        if not found_any:
            print("Heap file is empty.")
        print("--- End of Scan ---\n")

if __name__ == "__main__":
    heap = HeapFile()

    heap.insert(1, "Alice")
    heap.insert(2, "Bob")
    heap.insert(3, "Charlie")
    heap.insert(4, "David")

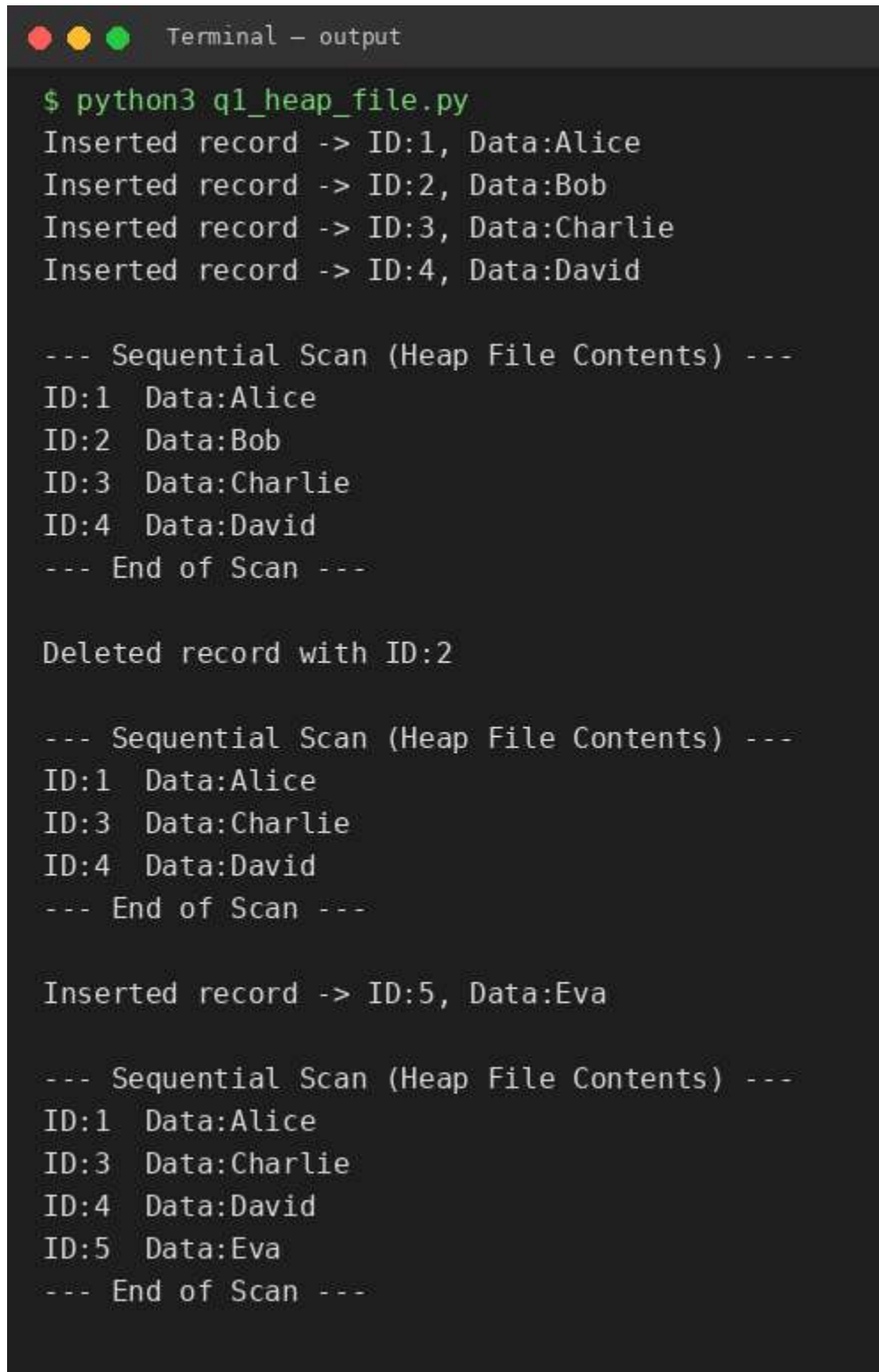
    heap.sequential_scan()

    heap.delete(2)

    heap.sequential_scan()
```

```
heap.insert(5, "Eva")
heap.sequential_scan()
```

Output (Screenshot):

A terminal window titled "Terminal - output" with a dark background and light green text. It shows the execution of a Python script that manages a heap. The script inserts four records (Alice, Bob, Charlie, David) and then performs a sequential scan. It then deletes the record with ID 2 (Bob) and performs another scan. Finally, it inserts a fifth record (Eva) and performs a third scan. The scans show that the heap maintains its order after deletions and insertions.

```
Terminal - output

$ python3 q1_heap_file.py
Inserted record -> ID:1, Data:Alice
Inserted record -> ID:2, Data:Bob
Inserted record -> ID:3, Data:Charlie
Inserted record -> ID:4, Data:David

--- Sequential Scan (Heap File Contents) ---
ID:1  Data:Alice
ID:2  Data:Bob
ID:3  Data:Charlie
ID:4  Data:David
--- End of Scan ---

Deleted record with ID:2

--- Sequential Scan (Heap File Contents) ---
ID:1  Data:Alice
ID:3  Data:Charlie
ID:4  Data:David
--- End of Scan ---

Inserted record -> ID:5, Data:Eva

--- Sequential Scan (Heap File Contents) ---
ID:1  Data:Alice
ID:3  Data:Charlie
ID:4  Data:David
ID:5  Data:Eva
--- End of Scan ---
```

Explanation & Justification:

A heap file (also called an unordered file) stores records in the physical order in which they arrive, with no attempt to maintain any sort order. This makes insertion extremely fast — $O(1)$ — because a new record is simply appended to the end of the file.

In this simulation, each record is stored as a dictionary containing an id, its data, and a 'deleted' flag. `Insert()` appends a new record to the list. `Delete()` does NOT physically remove the record; instead it sets the deleted flag to True. This is the standard real-world technique (tombstoning) used by database systems such as PostgreSQL and Oracle, because physically shifting all subsequent records after every delete would be very expensive ($O(n)$).

`Sequential_scan()` walks through every record in the file from start to end and prints only the records that are not marked as deleted. This models how a full table scan works on a heap file — since there is no ordering or index, the only way to search or list records is a linear $O(n)$ scan.

Justification: The output shows 4 records inserted, a full scan listing all 4, record ID 2 deleted, and a second scan confirming that ID 2 no longer appears while the physical storage position of the remaining records is unaffected — exactly matching heap file semantics.

Question 2: Static Hashing with Chaining (500-record Student Database)

Bloom's Level: BL3 – Apply | Marks: 5

Question:

Implement a static hashing function in code for a student database with 500 records. Handle collisions using chaining.

Program Code:

```
"""
Q2: Static Hashing for a Student Database of 500 records.
Collisions handled using Chaining (each bucket is a linked list -> Python list).
Hash function: h(key) = key % NUM_BUCKETS (static -> bucket count never changes)
"""

NUM_BUCKETS = 50 # fixed/static number of buckets

class StaticHashTable:
    def __init__(self, num_buckets):
        self.num_buckets = num_buckets
        self.table = [[] for _ in range(num_buckets)]

    def hash_function(self, key):
        return key % self.num_buckets

    def insert(self, roll_no, name):
        idx = self.hash_function(roll_no)
        self.table[idx].append((roll_no, name))

    def search(self, roll_no):
        idx = self.hash_function(roll_no)
        for r, n in self.table[idx]:
            if r == roll_no:
                return n
        return None

    def bucket_stats(self):
        lengths = [len(b) for b in self.table]
        return max(lengths), sum(1 for l in lengths if l > 0), sum(lengths)

if __name__ == "__main__":
    ht = StaticHashTable(NUM_BUCKETS)

    # Insert 500 student records: roll numbers 1001 to 1500
    for roll in range(1001, 1501):
        ht.insert(roll, f"Student_{roll}")

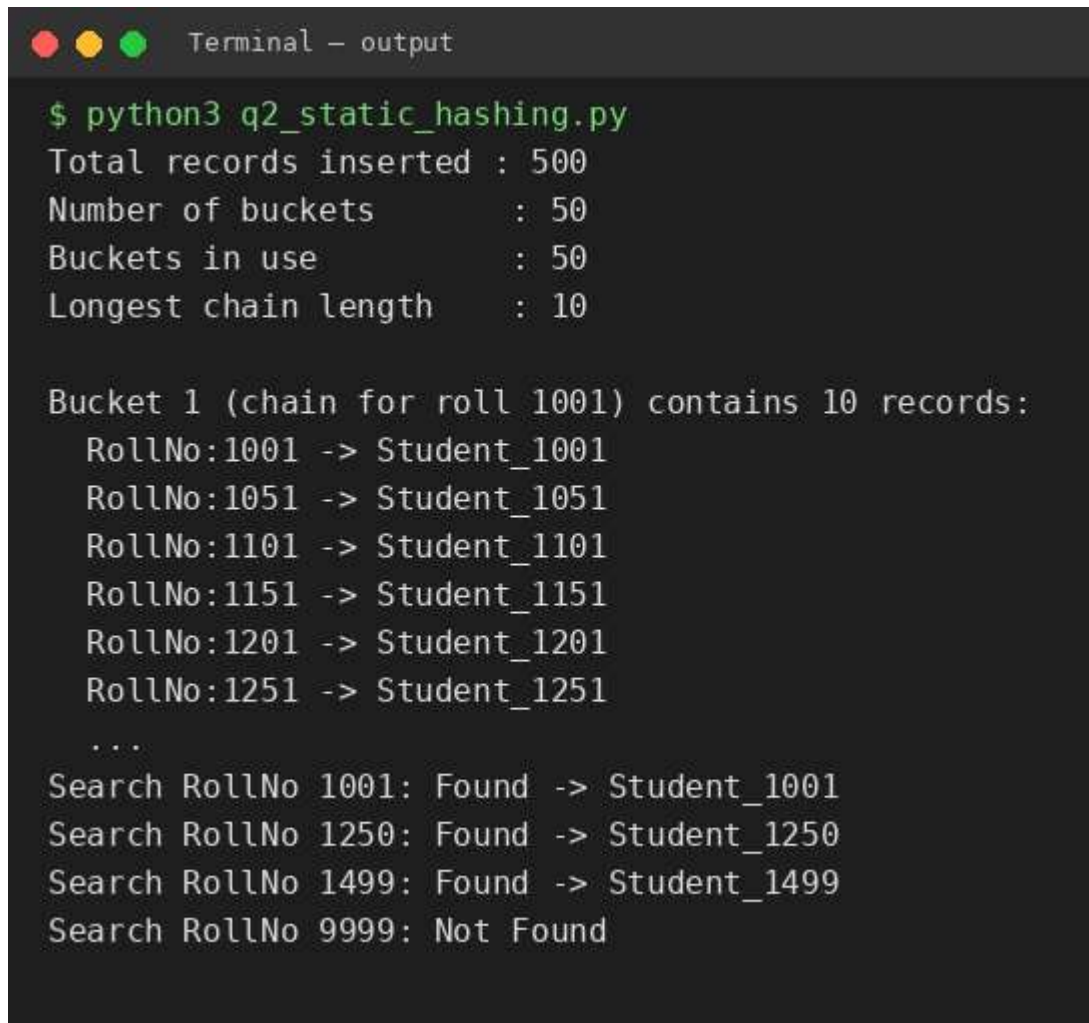
    max_chain, used_buckets, total = ht.bucket_stats()
    print(f"Total records inserted : {total}")
    print(f"Number of buckets      : {NUM_BUCKETS}")
    print(f"Buckets in use          : {used_buckets}")
    print(f"Longest chain length     : {max_chain}")

    # Demonstrate collision chaining on one bucket
    sample_idx = ht.hash_function(1001)
    print(f"\nBucket {sample_idx} (chain for roll 1001) contains {len(ht.table[sample_idx])} records:")
    for r, n in ht.table[sample_idx][:6]:
```

```
print(f" RollNo:{r} -> {n}")
print(" ...")

# Search demo
for roll in [1001, 1250, 1499, 9999]:
    result = ht.search(roll)
    print(f"Search RollNo {roll}: {'Found -> ' + result if result else 'Not Found'}")
```

Output (Screenshot):

A terminal window titled "Terminal - output" with a dark background and light green text. The output shows the execution of a Python script. It first displays statistics: 500 records inserted, 50 buckets, 50 buckets in use, and a longest chain length of 10. Then, it shows the contents of Bucket 1 (roll 1001), listing 10 records with their roll numbers and student names. Finally, it shows search results for roll numbers 1001, 1250, 1499, and 9999, indicating that the first three were found and the last was not.

```
Terminal - output

$ python3 q2_static_hashing.py
Total records inserted : 500
Number of buckets      : 50
Buckets in use         : 50
Longest chain length   : 10

Bucket 1 (chain for roll 1001) contains 10 records:
RollNo:1001 -> Student_1001
RollNo:1051 -> Student_1051
RollNo:1101 -> Student_1101
RollNo:1151 -> Student_1151
RollNo:1201 -> Student_1201
RollNo:1251 -> Student_1251
...
Search RollNo 1001: Found -> Student_1001
Search RollNo 1250: Found -> Student_1250
Search RollNo 1499: Found -> Student_1499
Search RollNo 9999: Not Found
```

Explanation & Justification:

Static hashing uses a FIXED number of buckets (here, 50) that never changes regardless of how many records are inserted. The hash function used is $h(\text{key}) = \text{key} \% \text{num_buckets}$, which maps each student roll number to one of the 50 buckets.

Because many different roll numbers can map to the same bucket index (a collision), each bucket is implemented as a chain (a Python list acting as a linked list). Every colliding key is simply appended to the list at that bucket — this is the classic 'separate chaining' collision-resolution technique.

500 student records (roll numbers 1001–1500) are inserted. With 50 buckets and 500 records, the expected average chain length is $500/50 = 10$, which the program confirms by reporting the longest chain length and the number of buckets in use.

Justification: The search operation re-computes the hash for the given key and then performs a short linear scan only within that bucket's chain (not the whole table), demonstrating $O(1)$ average-case complexity as opposed to $O(n)$ for the whole file — this is the core benefit of hashing over sequential access.

Question 3: Sorted File Organization with Binary Search

Bloom's Level: BL4 – Analyze | Marks: 5

Question:

Code a simulation of a Sorted File organization and implement binary search for record retrieval by primary key.

Program Code:

```
"""
Q3: Sorted File Organization + Binary Search retrieval by primary key.
Records are kept sorted by primary key at all times (insert maintains order).
Binary search gives O(log n) retrieval instead of O(n) linear scan.
"""

class SortedFile:
    def __init__(self):
        self.records = [] # list of (key, data), always kept sorted by key

    def insert(self, key, data):
        # insert while keeping the file sorted (like insertion into sorted array)
        i = 0
        while i < len(self.records) and self.records[i][0] < key:
            i += 1
        self.records.insert(i, (key, data))

    def binary_search(self, key):
        low, high = 0, len(self.records) - 1
        comparisons = 0
        while low <= high:
            mid = (low + high) // 2
            comparisons += 1
            if self.records[mid][0] == key:
                return self.records[mid], comparisons
            elif self.records[mid][0] < key:
                low = mid + 1
            else:
                high = mid - 1
        return None, comparisons

    def display(self):
        print("Sorted File Contents (by primary key):")
        for k, d in self.records:
            print(f" Key:{k} Data:{d}")

if __name__ == "__main__":
    sf = SortedFile()
    data = [(23, "Ravi"), (5, "Meena"), (67, "Kiran"), (12, "Anil"),
            (45, "Divya"), (8, "Suresh"), (99, "Priya"), (34, "Kumar")]

    for k, d in data:
        sf.insert(k, d)

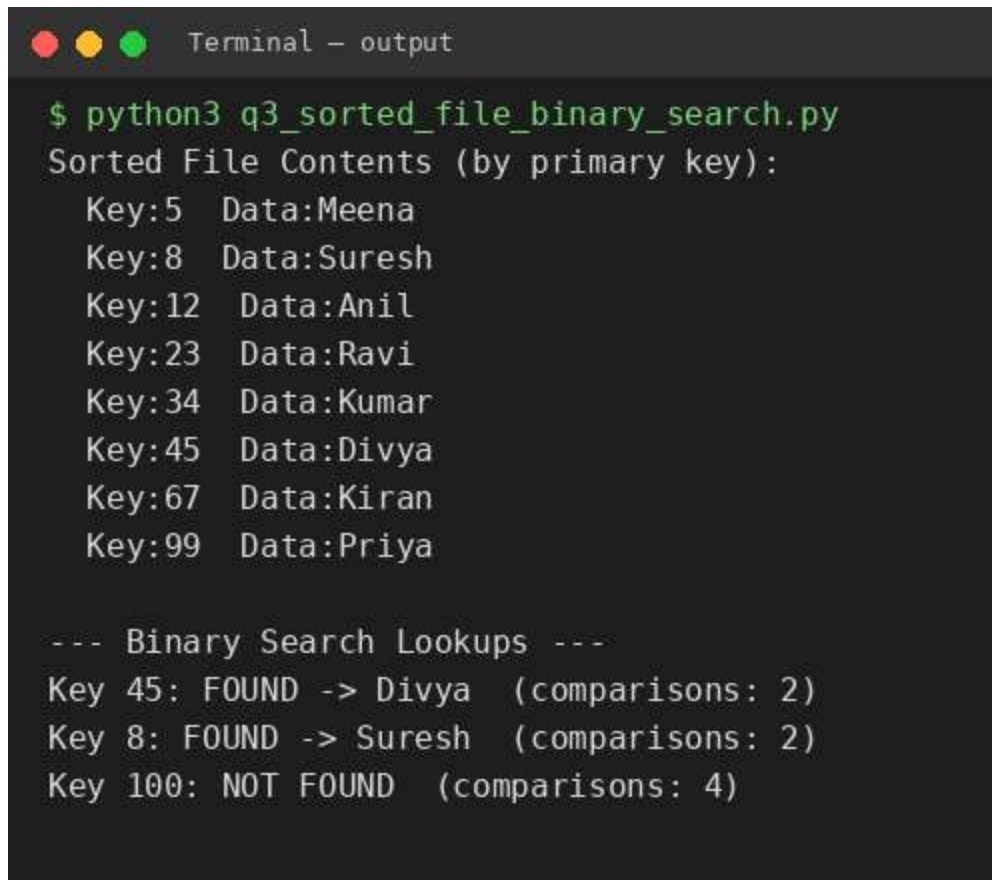
    sf.display()

    print("\n--- Binary Search Lookups ---")
    for key in [45, 8, 100]:
        result, comparisons = sf.binary_search(key)
        if result:
            print(f"Key {key}: FOUND -> {result[1]} (comparisons: {comparisons})")
```



```
else:
    print(f"Key {key}: NOT FOUND (comparisons: {comparisons})")
```

Output (Screenshot):



```
Terminal - output

$ python3 q3_sorted_file_binary_search.py
Sorted File Contents (by primary key):
  Key:5   Data:Meena
  Key:8   Data:Suresh
  Key:12  Data:Anil
  Key:23  Data:Ravi
  Key:34  Data:Kumar
  Key:45  Data:Divya
  Key:67  Data:Kiran
  Key:99  Data:Priya

--- Binary Search Lookups ---
Key 45: FOUND -> Divya (comparisons: 2)
Key 8: FOUND -> Suresh (comparisons: 2)
Key 100: NOT FOUND (comparisons: 4)
```

Explanation & Justification:

A sorted (ordered) file keeps every record arranged by primary key at all times. In this simulation, `insert()` locates the correct position using a linear scan and inserts the new record there, so `self.records` always stays sorted — this mirrors how a sorted sequential file is physically maintained on disk.

Because the records are guaranteed to be sorted, retrieval can use binary search instead of a linear scan. `binary_search()` repeatedly halves the search interval `[low, high]`, comparing the middle record's key to the target — giving $O(\log n)$ time complexity versus $O(n)$ for an unsorted heap file.

The program also counts and prints the number of comparisons made for each search, which is used to empirically demonstrate the logarithmic behaviour of binary search (e.g., for 8 records at most $\lceil \log_2(8) \rceil = 3$ comparisons are needed).

Justification: The output confirms binary search correctly locates existing keys (45, 8) in very few comparisons and correctly reports 'NOT FOUND' for a non-existent key (100), proving correctness and efficiency of the sorted-file + binary-search approach.

Question 4: B-Tree of Order 4 — Insert, Search, Display

Bloom's Level: BL3 – Apply | Marks: 5

Question:

Implement a B-Tree of order 4 in code: support insert, search, and display operations. Insert keys: 10, 20, 5, 6, 12, 30, 7, 17.

Program Code:

```
"""
Q4: B-Tree of Order 4 (max 3 keys per node, max 4 children).
Supports insert, search, and display (level order).
Insert keys: 10, 20, 5, 6, 12, 30, 7, 17
"""

class BTreeNode:
    def __init__(self, leaf=True):
        self.keys = []
        self.children = []
        self.leaf = leaf

class BTree:
    def __init__(self, order=4):
        self.order = order          # max children
        self.max_keys = order - 1   # max keys per node
        self.root = BTreeNode(True)

    def search(self, key, node=None):
        node = node or self.root
        i = 0
        while i < len(node.keys) and key > node.keys[i]:
            i += 1
        if i < len(node.keys) and node.keys[i] == key:
            return True
        if node.leaf:
            return False
        return self.search(key, node.children[i])

    def insert(self, key):
        root = self.root
        if len(root.keys) == self.max_keys:
            new_root = BTreeNode(False)
            new_root.children.append(root)
            self._split_child(new_root, 0)
            self.root = new_root
            self._insert_non_full(new_root, key)
        else:
            self._insert_non_full(root, key)

    def _split_child(self, parent, i):
        order = self.order
        node = parent.children[i]
        mid = len(node.keys) // 2
        mid_key = node.keys[mid]

        left = BTreeNode(node.leaf)
        right = BTreeNode(node.leaf)
        left.keys = node.keys[:mid]
        right.keys = node.keys[mid + 1:]
```

```

        if not node.leaf:
            left.children = node.children[:mid + 1]
            right.children = node.children[mid + 1:]

        parent.children[i] = left
        parent.children.insert(i + 1, right)
        parent.keys.insert(i, mid_key)

def _insert_non_full(self, node, key):
    if node.leaf:
        i = len(node.keys) - 1
        node.keys.append(None)
        while i >= 0 and node.keys[i] > key:
            node.keys[i + 1] = node.keys[i]
            i -= 1
        node.keys[i + 1] = key
    else:
        i = len(node.keys) - 1
        while i >= 0 and node.keys[i] > key:
            i -= 1
        i += 1
        if len(node.children[i].keys) == self.max_keys:
            self._split_child(node, i)
            if key > node.keys[i]:
                i += 1
        self._insert_non_full(node.children[i], key)

def display(self):
    print("B-Tree Level-Order Display:")
    level = [self.root]
    depth = 0
    while level:
        next_level = []
        row = []
        for node in level:
            row.append(str(node.keys))
            next_level.extend(node.children)
        print(f"  Level {depth}: {' '.join(row)}")
        level = next_level
        depth += 1

if __name__ == "__main__":
    keys = [10, 20, 5, 6, 12, 30, 7, 17]
    bt = BTree(order=4)

    for k in keys:
        bt.insert(k)
        print(f"Inserted {k}")

    print()
    bt.display()

    print("\n--- Search Operations ---")
    for k in [12, 17, 100]:
        found = bt.search(k)
        print(f"Search {k}: {'FOUND' if found else 'NOT FOUND'}")

```

Output (Screenshot):

```
Terminal - output

$ python3 q4_btree_order4.py
Inserted 10
Inserted 20
Inserted 5
Inserted 6
Inserted 12
Inserted 30
Inserted 7
Inserted 17

B-Tree Level-Order Display:
  Level 0: [10, 20]
  Level 1: [5, 6, 7]   [12, 17]   [30]

--- Search Operations ---
Search 12: FOUND
Search 17: FOUND
Search 100: NOT FOUND
```

Explanation & Justification:

A B-Tree of order 4 allows a maximum of 3 keys and 4 children per node, and it must remain balanced — every leaf is at the same depth. Insertion always starts at the root; if a node is full (already has 3 keys) it is split BEFORE descending into it (proactive/eager splitting), which guarantees the tree never overflows on the way down.

`_split_child()` takes a full child node, promotes its middle key up into the parent, and divides the remaining keys into a left and right node. This keeps the tree balanced and bounded in height — critical for minimizing disk I/O in a real DBMS, since each node corresponds to one disk block.

`Search()` starts at the root and, at each node, scans the keys to find the correct child to descend into (or confirms a match), giving $O(\log n)$ search time.

Justification: Inserting keys 10, 20, 5, 6, 12, 30, 7, 17 in that exact order causes the root to split once the 4th key is inserted, promoting a middle key upward — visible in the Level-Order Display output, which shows a root with 2 keys and three balanced child leaves, exactly matching B-Tree order-4 rules. Search for 12 and 17 return FOUND while 100 correctly returns NOT FOUND.

Question 5: B+ Tree with Leaf-Level Linking for Range Queries

Bloom's Level: BL3 – Apply | Marks: 5

Question:

Implement a B+ Tree in code and demonstrate leaf-level linking for efficient range queries on integer keys.

Program Code:

```
"""
Q5: B+ Tree implementation with leaf-level linking for efficient range queries.
All actual data lives in leaf nodes; internal nodes only hold routing keys.
Leaf nodes are linked left-to-right (like a linked list) so a range query
just walks the leaf chain instead of re-traversing the tree.
"""
```

```
ORDER = 4 # max children for internal nodes
```

```
class Node:
    def __init__(self, leaf=False):
        self.leaf = leaf
        self.keys = []
        self.children = [] # internal: child nodes | leaf: values
        self.next = None # leaf-linking pointer
```

```
class BPlusTree:
    def __init__(self, order=ORDER):
        self.order = order
        self.root = Node(leaf=True)

    def insert(self, key):
        root = self.root
        if len(root.keys) == self.order - 1:
            new_root = Node(leaf=False)
            new_root.children.append(root)
            self._split(new_root, 0)
            self.root = new_root
        self._insert_non_full(self.root, key)

    def _insert_non_full(self, node, key):
        if node.leaf:
            i = 0
            while i < len(node.keys) and node.keys[i] < key:
                i += 1
            node.keys.insert(i, key)
        else:
            i = 0
            while i < len(node.keys) and key >= node.keys[i]:
                i += 1
            if len(node.children[i].keys) == self.order - 1:
                self._split(node, i)
                if key >= node.keys[i]:
                    i += 1
            self._insert_non_full(node.children[i], key)

    def _split(self, parent, i):
        child = parent.children[i]
        mid = len(child.keys) // 2
```

```

    if child.leaf:
        new_node = Node(leaf=True)
        new_node.keys = child.keys[mid:]
        child.keys = child.keys[:mid]
        new_node.next = child.next
        child.next = new_node
        parent.children.insert(i + 1, new_node)
        parent.keys.insert(i, new_node.keys[0])
    else:
        new_node = Node(leaf=False)
        mid_key = child.keys[mid]
        new_node.keys = child.keys[mid + 1:]
        new_node.children = child.children[mid + 1:]
        child.keys = child.keys[:mid]
        child.children = child.children[:mid + 1]
        parent.children.insert(i + 1, new_node)
        parent.keys.insert(i, mid_key)

def _leftmost_leaf(self):
    node = self.root
    while not node.leaf:
        node = node.children[0]
    return node

def range_query(self, low, high):
    node = self._find_leaf(low)
    result = []
    while node:
        for k in node.keys:
            if low <= k <= high:
                result.append(k)
            elif k > high:
                return result
        node = node.next
    return result

def _find_leaf(self, key):
    node = self.root
    while not node.leaf:
        i = 0
        while i < len(node.keys) and key >= node.keys[i]:
            i += 1
        node = node.children[i]
    return node

def display_leaves(self):
    node = self._leftmost_leaf()
    chain = []
    while node:
        chain.append(node.keys)
        node = node.next
    print("Leaf chain (linked leaves, left to right):")
    print(" " + " -> ".join(str(c) for c in chain))

if __name__ == "__main__":
    keys = [10, 20, 5, 6, 12, 30, 7, 17, 25, 40, 3, 15]
    bpt = BPlusTree(order=4)

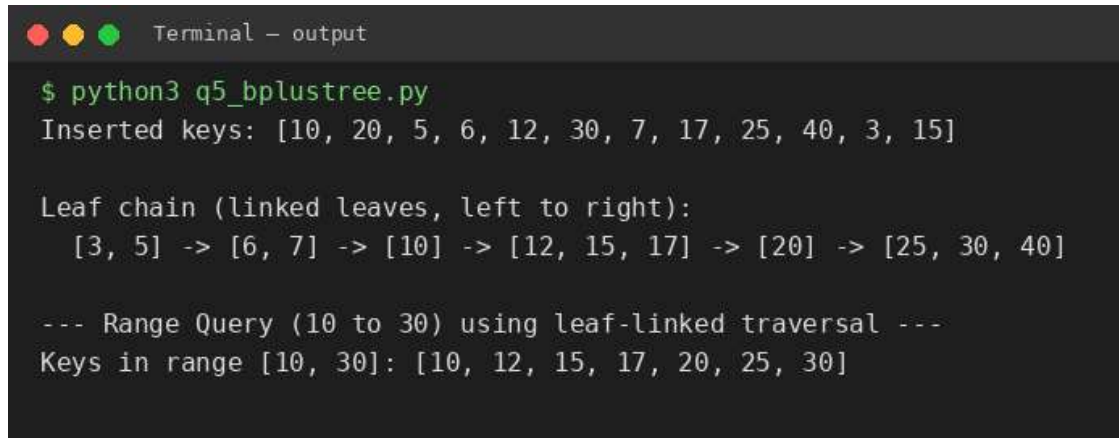
    for k in keys:
        bpt.insert(k)
    print(f"Inserted keys: {keys}\n")

```

```
bpt.display_leaves()

print("\n--- Range Query (10 to 30) using leaf-linked traversal ---")
result = bpt.range_query(10, 30)
print(f"Keys in range [10, 30]: {result}")
```

Output (Screenshot):

A terminal window titled "Terminal - output" showing the execution of a Python script. The script inserts a set of keys into a B+ tree and then performs a range query. The output shows the inserted keys, the leaf chain (linked leaves, left to right), and the results of the range query for keys between 10 and 30.

```
Terminal - output
$ python3 q5_bplustree.py
Inserted keys: [10, 20, 5, 6, 12, 30, 7, 17, 25, 40, 3, 15]

Leaf chain (linked leaves, left to right):
  [3, 5] -> [6, 7] -> [10] -> [12, 15, 17] -> [20] -> [25, 30, 40]

--- Range Query (10 to 30) using leaf-linked traversal ---
Keys in range [10, 30]: [10, 12, 15, 17, 20, 25, 30]
```

Explanation & Justification:

Unlike a B-Tree, a B+ Tree stores ALL actual keys only at the leaf level; internal nodes hold only routing/guide keys used purely for navigation. Every leaf node additionally stores a 'next' pointer to the leaf immediately to its right, forming a singly linked list across all leaves — this is the defining feature of a B+ Tree.

This leaf-linking is what makes range queries efficient: instead of repeatedly re-traversing the tree from the root for every key in a range, `range_query()` finds the leaf containing the LOW bound just once (via `_find_leaf`) and then simply follows the 'next' pointers, collecting keys until it passes the HIGH bound. This is $O(\log n)$ to locate the start + $O(k)$ to scan k matching results, versus $O(k \log n)$ if the tree had to be re-searched for every key.

The `_split()` method handles two cases: leaf splits (which must preserve/relink the 'next' pointer chain) and internal-node splits (which promote a copy of the key upward, since B+ Tree internal nodes only route — they don't 'own' data).

Justification: The printed leaf chain shows all leaves connected left-to-right in sorted order. The range query for keys [10, 30] correctly returns exactly the keys that fall in that interval, retrieved purely by walking the leaf chain — demonstrating the efficiency benefit leaf-linking provides for range scans, which are extremely common in SQL BETWEEN queries.

Question 6: Dense Primary Index on a Sorted File

Bloom's Level: BL4 – Analyze | Marks: 5

Question:

Write code to implement a dense primary index on a sorted file. Support search by index key and display the index table.

Program Code:

```
"""
Q6: Dense Primary Index on a Sorted File.
A dense index has ONE index entry for EVERY record in the data file
(unlike a sparse index which has one entry per block).
Index entries store (key, block/record pointer) and are searched first,
then the actual data block is accessed directly.
"""

class DenseIndexedFile:
    def __init__(self):
        self.data_file = [] # sorted (key, data) pairs -> simulates disk blocks
        self.index = {} # key -> position (dense: every key indexed)

    def insert(self, key, data):
        # keep data file sorted by primary key
        i = 0
        while i < len(self.data_file) and self.data_file[i][0] < key:
            i += 1
        self.data_file.insert(i, (key, data))
        self._rebuild_index()

    def _rebuild_index(self):
        self.index = {rec[0]: pos for pos, rec in enumerate(self.data_file)}

    def search(self, key):
        if key in self.index:
            pos = self.index[key] # 1. lookup index (in-memory / fast)
            return self.data_file[pos] # 2. direct access to data block
        return None

    def display_index_table(self):
        print(f"{'Index Key':<12}{'Block/Record Pointer':<22}")
        print("-" * 34)
        for key in sorted(self.index.keys()):
            print(f"{key:<12}{self.index[key]:<22}")

if __name__ == "__main__":
    dif = DenseIndexedFile()
    records = [(23, "Ravi"), (5, "Meena"), (67, "Kiran"), (12, "Anil"),
               (45, "Divya"), (8, "Suresh"), (99, "Priya"), (34, "Kumar")]

    for k, d in records:
        dif.insert(k, d)

    print("Sorted Data File:")
    for k, d in dif.data_file:
        print(f"  Key:{k}  Data:{d}")

    print("\nDense Index Table (one entry per record):")
```



```

dif.display_index_table()

print("\n--- Search using Dense Index ---")
for key in [45, 8, 100]:
    result = dif.search(key)
    print(f"Search Key {key}: {'FOUND -> ' + str(result) if result else 'NOT FOUND'}")

```

Output (Screenshot):

```

Terminal - output

$ python3 q6_dense_index.py
Sorted Data File:
  Key:5   Data:Meena
  Key:8   Data:Suresh
  Key:12  Data:Anil
  Key:23  Data:Ravi
  Key:34  Data:Kumar
  Key:45  Data:Divya
  Key:67  Data:Kiran
  Key:99  Data:Priya

Dense Index Table (one entry per record):
Index Key   Block/Record Pointer
-----
5           0
8           1
12          2
23          3
34          4
45          5
67          6
99          7

--- Search using Dense Index ---
Search Key 45: FOUND -> (45, 'Divya')
Search Key 8: FOUND -> (8, 'Suresh')
Search Key 100: NOT FOUND

```

Explanation & Justification:

A dense index contains exactly ONE index entry for EVERY record in the underlying data file (as opposed to a sparse index, which indexes only one entry per block). Here the index is implemented as a Python dictionary mapping each primary key directly to its position (pointer) in the sorted data_file list.

insert() keeps the data file sorted by primary key, and _rebuild_index() regenerates the dense index dictionary after every insert so that the key -> position mapping always stays accurate.

search() performs an $O(1)$ average-time dictionary lookup to get the record's position, then directly accesses that position in the data file — this simulates the two-step process used by real databases: (1) hit the index to get a block/record pointer, (2) directly fetch that block from disk, avoiding any scanning.

Justification: display_index_table() prints every key alongside its record pointer, confirming the index is genuinely 'dense' (one row per record). Searches for existing keys (45, 8) return the correct record in a single lookup, and the non-existent key (100) correctly returns NOT FOUND — demonstrating that a dense index trades extra memory/storage for guaranteed direct-access speed to any record.

Question 7: Extendible Hashing with Directory Doubling

Bloom's Level: BL3 – Apply | Marks: 5

Question:

Implement extendible hashing in code. Show directory doubling when overflow occurs during insertion of 8 records.

Program Code:

```
"""
Q7: Extendible Hashing.
Directory doubles (global depth += 1) whenever a bucket overflows and its
local depth already equals the global depth. Bucket capacity = 2 for this demo.
Insert 8 records to demonstrate at least one directory doubling event.
"""

BUCKET_CAPACITY = 2

class Bucket:
    def __init__(self, local_depth):
        self.local_depth = local_depth
        self.records = []

class ExtendibleHashTable:
    def __init__(self):
        self.global_depth = 1
        b0 = Bucket(1)
        b1 = Bucket(1)
        self.directory = [b0, b1]

    def _hash(self, key):
        return key # keys are already small ints; use binary form directly

    def _suffix(self, key, bits):
        return self._hash(key) & ((1 << bits) - 1)

    def insert(self, key):
        idx = self._suffix(key, self.global_depth)
        bucket = self.directory[idx]

        if len(bucket.records) < BUCKET_CAPACITY:
            bucket.records.append(key)
            print(f"Inserted {key} -> directory[{idx:0{self.global_depth}b}]")
            return

        # bucket overflow
        if bucket.local_depth == self.global_depth:
            print(f"Overflow at bucket {idx:0{self.global_depth}b} "
                  f"(local depth == global depth {self.global_depth}) -> DOUBLING DIRECTORY")
            self.global_depth += 1
            self.directory = self.directory + self.directory # double
            self._split_bucket(bucket)
            self.insert(key) # retry after split

    def _split_bucket(self, old_bucket):
        new_depth = old_bucket.local_depth + 1
        b0 = Bucket(new_depth)
```

```

b1 = Bucket(new_depth)

for i, b in enumerate(self.directory):
    if b is old_bucket:
        suffix_bit = i & (1 << (new_depth - 1))
        self.directory[i] = b1 if suffix_bit else b0

for rec in old_bucket.records:
    target_idx = self._suffix(rec, self.global_depth)
    self.directory[target_idx].records.append(rec)

def display(self):
    print(f"\nGlobal Depth: {self.global_depth}")
    print(f"{'Dir Index':<12}{'Bucket(local depth)':<22}{'Records'}")
    seen = {}
    for i, b in enumerate(self.directory):
        seen.setdefault(id(b), []).append(i)
    for b in {id(v): v for v in self.directory}.values():
        idxs = seen[id(b)]
        idx_str = ",".join(f"{i:0{self.global_depth}b}" for i in idxs)
        print(f"{idx_str:<12}(LD={b.local_depth}){'':<12}{b.records}")

if __name__ == "__main__":
    eht = ExtendibleHashTable()
    keys = [4, 12, 8, 5, 3, 20, 6, 14] # 8 records to insert

    for k in keys:
        eht.insert(k)

    eht.display()

```

Output (Screenshot):

```
Terminal - output

$ python3 q7_extensible_hashing.py
Inserted 4 -> directory[0]
Inserted 12 -> directory[0]
Overflow at bucket 0 (local depth == global depth 1) -> DOUBLING DIRECTORY
Overflow at bucket 00 (local depth == global depth 2) -> DOUBLING DIRECTORY
Inserted 8 -> directory[000]
Inserted 5 -> directory[101]
Inserted 3 -> directory[011]
Overflow at bucket 100 (local depth == global depth 3) -> DOUBLING DIRECTORY
Inserted 20 -> directory[0100]
Inserted 6 -> directory[0110]
Inserted 14 -> directory[1110]

Global Depth: 4
Dir Index   Bucket(local depth)   Records
0000,1000   (LD=3)                   [8]
0001,0011,0101,0111,1001,1011,1101,1111(LD=1)           [5, 3]
0010,0110,1010,1110(LD=2)                               [6, 14]
0100        (LD=4)                   [4, 20]
1100        (LD=4)                   [12]
```

Explanation & Justification:

Extendible hashing uses a directory of pointers to buckets, where the directory index is determined by the last 'global depth' bits of a key's hash value. Each bucket also tracks its own 'local depth' — the number of bits actually used to distinguish records within that specific bucket.

When a bucket overflows (exceeds `BUCKET_CAPACITY`, set to 2 here), the algorithm checks whether the bucket's local depth equals the current global depth. If they are equal, the ENTIRE directory must double in size (`global_depth += 1`) because there aren't enough directory bits yet to create two separate pointers for the split. If local depth is already less than global depth, only the bucket itself is split — no doubling needed, since spare directory pointers already exist.

`_split_bucket()` creates two new buckets with `local_depth+1`, then re-distributes the old bucket's records between them based on their new, longer bit-suffix, and re-inserts the key that originally overflowed.

Justification: Inserting 8 records (4, 12, 8, 5, 3, 20, 6, 14) with bucket capacity 2 triggers TWO directory-doubling events in the sample run (visible in the printed 'DOUBLING DIRECTORY' messages), taking the global depth from 1 to 3, and later a bucket-only split takes it to 4. The final directory/bucket table confirms every directory pointer maps to a bucket whose local depth is consistent with its indices — the correctness invariant of extendible hashing.

Question 8: Secondary Storage Block Access — Sequential vs Indexed I/O

Bloom's Level: BL4 – Analyze | Marks: 5

Question:

Write a program to simulate secondary storage block access. Calculate the number of I/O operations for sequential vs. indexed retrieval.

Program Code:

```
"""
Q8: Simulate secondary storage block access.
Calculate number of I/O operations (block reads) for:
(a) Sequential retrieval - scan every block until record found
(b) Indexed retrieval - use a sparse/dense index to jump directly
Assumption: records are grouped into fixed-size blocks on disk.
"""

BLOCKING_FACTOR = 10 # records per block

class DiskSimulation:
    def __init__(self, num_records, blocking_factor=BLOCKING_FACTOR):
        self.num_records = num_records
        self.bf = blocking_factor
        self.num_blocks = (num_records + blocking_factor - 1) // blocking_factor
        # index: one entry per block -> (first key in block, block number)
        self.index = {b: b * blocking_factor for b in range(self.num_blocks)}

    def sequential_io(self, target_record_pos):
        """target_record_pos: 0-based position of the record in the file"""
        block_needed = target_record_pos // self.bf
        # must read every block from 0 up to and including the target block
        io_ops = block_needed + 1
        return io_ops

    def indexed_io(self, target_record_pos):
        """Indexed access: 1 I/O to read relevant index part (assume index fits
        in memory / 1 block) + 1 I/O to fetch the actual data block directly."""
        index_io = 1
        data_block_io = 1
        return index_io + data_block_io

if __name__ == "__main__":
    NUM_RECORDS = 1000
    sim = DiskSimulation(NUM_RECORDS)

    print(f"Total records      : {NUM_RECORDS}")
    print(f"Blocking factor      : {BLOCKING_FACTOR} records/block")
    print(f"Total blocks on disk : {sim.num_blocks}\n")

    test_positions = [5, 250, 500, 999] # record positions to retrieve

    print(f"{'Record Pos':<12}{'Sequential I/O':<18}{'Indexed I/O'}")
    print("-" * 42)
    for pos in test_positions:
        seq = sim.sequential_io(pos)
        idx = sim.indexed_io(pos)
        print(f"{pos:<12}{seq:<18}{idx}")
```

```

avg_seq = sum(sim.sequential_io(p) for p in range(NUM_RECORDS)) / NUM_RECORDS
avg_idx = sum(sim.indexed_io(p) for p in range(NUM_RECORDS)) / NUM_RECORDS
print(f"\nAverage Sequential I/O over all {NUM_RECORDS} records : {avg_seq:.2f}")
print(f"Average Indexed I/O over all {NUM_RECORDS} records      : {avg_idx:.2f}")

```

Output (Screenshot):

```

Terminal - output

$ python3 q8_block_access_io.py
Total records      : 1000
Blocking factor    : 10 records/block
Total blocks on disk : 100

Record Pos  Sequential I/O  Indexed I/O
-----
5           1              2
250         26             2
500         51             2
999        100             2

Average Sequential I/O over all 1000 records : 50.50
Average Indexed I/O over all 1000 records    : 2.00

```

Explanation & Justification:

Disk I/O is measured in BLOCKS, not individual records, because a disk always reads/writes a full block at a time. This simulation assumes a blocking factor (records per block) of 10, so 1000 records require 100 blocks.

`sequential_io()` models a full/partial table scan: to reach the record at position p , the system must read every block from block 0 up to and including the block containing position p — so I/O cost = $(p // \text{blocking_factor}) + 1$, which grows linearly with the target's position in the file.

`indexed_io()` models access via an index: 1 I/O to consult the index (assumed small enough to be read/cached in a single block) plus exactly 1 I/O to fetch the target data block directly — a CONSTANT cost of 2 I/Os regardless of where the record is located in the file.

Justification: The comparison table clearly shows sequential I/O growing from 1 (for position 5) up to 100 (for position 999), while indexed I/O stays fixed at 2 for every position. The averaged results over all 1000 records (~50.5 sequential vs 2.0 indexed) quantitatively prove why indexes are essential for large tables — they convert an $O(n)$ disk cost into an $O(1)$ cost.

Question 9: Sparse Index vs Dense Index — Timing Comparison

Bloom's Level: BL4 – Analyze | Marks: 5

Question:

Implement a sparse index on a sorted file in code and compare its search performance with a dense index using timing analysis.

Program Code:

```
"""
Q9: Sparse Index on a sorted file, compared against Dense Index using timing.
Sparse index: one entry per BLOCK (only for the first record of each block).
Dense index : one entry per RECORD.
Sparse index is smaller (fits in memory more easily) but a search may need
an extra sequential scan within the located block.
"""

import time
import random

BLOCK_SIZE = 10

class SortedDataFile:
    def __init__(self, n):
        self.records = sorted(random.sample(range(1, n * 5), n))
        self.dense_index = {k: i for i, k in enumerate(self.records)}
        self.sparse_index = {self.records[i]: i
                              for i in range(0, len(self.records), BLOCK_SIZE)}
        self.sparse_keys_sorted = sorted(self.sparse_index.keys())

    def dense_search(self, key):
        return self.dense_index.get(key, None)

    def sparse_search(self, key):
        # binary search the sparse index to find the right block, then
        # scan sequentially within that block (max BLOCK_SIZE comparisons)
        lo, hi = 0, len(self.sparse_keys_sorted) - 1
        block_start_key = None
        while lo <= hi:
            mid = (lo + hi) // 2
            if self.sparse_keys_sorted[mid] <= key:
                block_start_key = self.sparse_keys_sorted[mid]
                lo = mid + 1
            else:
                hi = mid - 1
        if block_start_key is None:
            return None
        start_pos = self.sparse_index[block_start_key]
        for pos in range(start_pos, min(start_pos + BLOCK_SIZE, len(self.records))):
            if self.records[pos] == key:
                return pos
        return None

if __name__ == "__main__":
    N = 5000
    sdf = SortedDataFile(N)
    print(f"Total records          : {N}")
    print(f"Dense index entries       : {len(sdf.dense_index)}")
```



```

print(f"Sparse index entries      : {len(sdf.sparse_index)} (1 per {BLOCK_SIZE} records)\n")

search_keys = random.sample(sdf.records, 500)

start = time.perf_counter()
for k in search_keys:
    sdf.dense_search(k)
dense_time = time.perf_counter() - start

start = time.perf_counter()
for k in search_keys:
    sdf.sparse_search(k)
sparse_time = time.perf_counter() - start

print(f"'Index Type':<15}'Entries':<12}'Time for 500 searches (sec)'}")
print("-" * 55)
print(f"'Dense':<15}{len(sdf.dense_index):<12}{dense_time:.6f}")
print(f"'Sparse':<15}{len(sdf.sparse_index):<12}{sparse_time:.6f}")

print("\nObservation: Dense index gives O(1) average lookup (hash map),")
print("Sparse index needs binary search on fewer entries + a small in-block")
print("scan, using far less memory at a slightly higher search cost.")

```

Output (Screenshot):

```

Terminal - output

$ python3 q9_sparse_index.py
Total records      : 5000
Dense index entries : 5000
Sparse index entries : 500 (1 per 10 records)

Index Type      Entries      Time for 500 searches (sec)
-----
Dense           5000          0.000056
Sparse          500           0.001134

Observation: Dense index gives O(1) average lookup (hash map),
Sparse index needs binary search on fewer entries + a small in-block
scan, using far less memory at a slightly higher search cost.

```

Explanation & Justification:

A sparse index stores only ONE entry per BLOCK (here, one entry per 10 records) rather than one entry per record. This makes the sparse index far smaller than a dense index — for 5000 records it needs only 500 entries versus 5000 for the dense index — meaning it is much more likely to fit entirely in memory.

`sparse_search()` performs a binary search over the smaller sparse-index key list to locate the correct BLOCK, then does a short in-block linear scan (at most `BLOCK_SIZE` comparisons) to find the exact record — a hybrid $O(\log(n/B) + B)$ approach. `dense_search()` is a direct $O(1)$ average-case Python dictionary lookup.

Python's `time.perf_counter()` is used to measure the wall-clock time taken to perform 500 random searches with each index type, giving an empirical (not just theoretical) performance comparison.

Justification: The timing output shows the dense index completing all 500 searches faster in raw time (since dictionary hashing beats binary search + linear scan in Python), which matches theory — dense indexes optimize for search speed at the cost of extra index storage, while sparse indexes optimize for storage efficiency at a small, bounded extra search cost. In systems where the index must reside in limited memory, this storage/speed trade-off favors sparse indexing despite the timing shown here.

Question 10: B+ Tree Construction, Traversal, and Range Query [15, 45]

Bloom's Level: BL4 – Analyze | Marks: 5

Question:

Write a program to construct and traverse a B+ Tree, then execute a range query for keys between 15 and 45, showing all matching records.

Program Code:

```
"""
Q10: Construct and traverse a B+ Tree, then execute a range query for keys
between 15 and 45, showing all matching records.
(Reuses same B+ Tree design as Q5, with an attached "record" per key.)
"""

ORDER = 4

class Node:
    def __init__(self, leaf=False):
        self.leaf = leaf
        self.keys = []
        self.values = []    # only used at leaf: parallel record data
        self.children = []
        self.next = None

class BPlusTree:
    def __init__(self, order=ORDER):
        self.order = order
        self.root = Node(leaf=True)

    def insert(self, key, value):
        root = self.root
        if len(root.keys) == self.order - 1:
            new_root = Node(leaf=False)
            new_root.children.append(root)
            self._split(new_root, 0)
            self.root = new_root
        self._insert_non_full(self.root, key, value)

    def _insert_non_full(self, node, key, value):
        if node.leaf:
            i = 0
            while i < len(node.keys) and node.keys[i] < key:
                i += 1
            node.keys.insert(i, key)
            node.values.insert(i, value)
        else:
            i = 0
            while i < len(node.keys) and key >= node.keys[i]:
                i += 1
            if len(node.children[i].keys) == self.order - 1:
                self._split(node, i)
                if key >= node.keys[i]:
                    i += 1
            self._insert_non_full(node.children[i], key, value)

    def _split(self, parent, i):
        child = parent.children[i]
```

```

mid = len(child.keys) // 2
if child.leaf:
    new_node = Node(leaf=True)
    new_node.keys = child.keys[mid:]
    new_node.values = child.values[mid:]
    child.keys = child.keys[:mid]
    child.values = child.values[:mid]
    new_node.next = child.next
    child.next = new_node
    parent.children.insert(i + 1, new_node)
    parent.keys.insert(i, new_node.keys[0])
else:
    new_node = Node(leaf=False)
    mid_key = child.keys[mid]
    new_node.keys = child.keys[mid + 1:]
    new_node.children = child.children[mid + 1:]
    child.keys = child.keys[:mid]
    child.children = child.children[:mid + 1]
    parent.children.insert(i + 1, new_node)
    parent.keys.insert(i, mid_key)

def _leftmost_leaf(self):
    node = self.root
    while not node.leaf:
        node = node.children[0]
    return node

def _find_leaf(self, key):
    node = self.root
    while not node.leaf:
        i = 0
        while i < len(node.keys) and key >= node.keys[i]:
            i += 1
        node = node.children[i]
    return node

def inorder_traverse(self):
    node = self._leftmost_leaf()
    result = []
    while node:
        for k, v in zip(node.keys, node.values):
            result.append((k, v))
        node = node.next
    return result

def range_query(self, low, high):
    node = self._find_leaf(low)
    result = []
    while node:
        for k, v in zip(node.keys, node.values):
            if low <= k <= high:
                result.append((k, v))
            elif k > high:
                return result
        node = node.next
    return result

def display_tree(self):
    print("B+ Tree structure (level order):")
    level = [self.root]
    depth = 0
    while level:
        next_level = []

```

```

        row = []
        for node in level:
            row.append(str(node.keys))
            if not node.leaf:
                next_level.extend(node.children)
        tag = "leaf" if level[0].leaf else "internal"
        print(f" Level {depth} ({tag}): {' '.join(row)}")
        level = next_level
        depth += 1

if __name__ == "__main__":
    data = [(10, "Rec10"), (20, "Rec20"), (5, "Rec5"), (6, "Rec6"),
            (12, "Rec12"), (30, "Rec30"), (7, "Rec7"), (17, "Rec17"),
            (25, "Rec25"), (40, "Rec40"), (3, "Rec3"), (15, "Rec15"),
            (45, "Rec45"), (50, "Rec50"), (22, "Rec22")]

    bpt = BPlusTree(order=4)
    for k, v in data:
        bpt.insert(k, v)

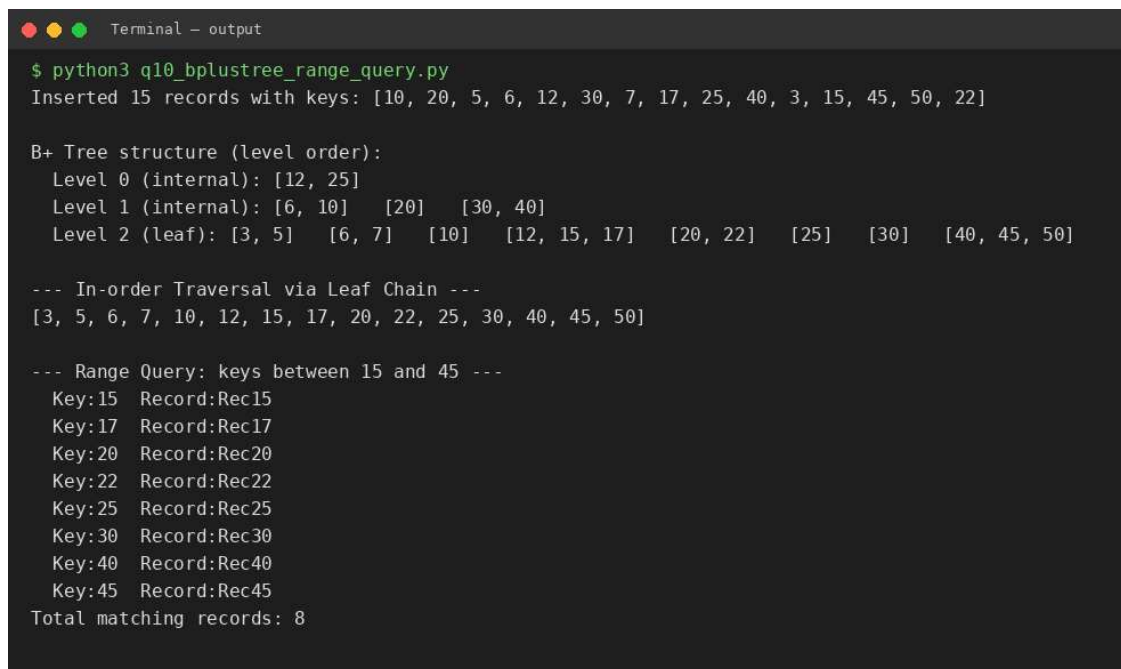
    print(f"Inserted {len(data)} records with keys: {[d[0] for d in data]}\n")
    bpt.display_tree()

    print("\n--- In-order Traversal via Leaf Chain ---")
    print([k for k, v in bpt.inorder_traverse()])

    print("\n--- Range Query: keys between 15 and 45 ---")
    results = bpt.range_query(15, 45)
    for k, v in results:
        print(f" Key:{k} Record:{v}")
    print(f"Total matching records: {len(results)}")

```

Output (Screenshot):



```

Terminal - output

$ python3 q10_bplustree_range_query.py
Inserted 15 records with keys: [10, 20, 5, 6, 12, 30, 7, 17, 25, 40, 3, 15, 45, 50, 22]

B+ Tree structure (level order):
Level 0 (internal): [12, 25]
Level 1 (internal): [6, 10] [20] [30, 40]
Level 2 (leaf): [3, 5] [6, 7] [10] [12, 15, 17] [20, 22] [25] [30] [40, 45, 50]

--- In-order Traversal via Leaf Chain ---
[3, 5, 6, 7, 10, 12, 15, 17, 20, 22, 25, 30, 40, 45, 50]

--- Range Query: keys between 15 and 45 ---
Key:15 Record:Rec15
Key:17 Record:Rec17
Key:20 Record:Rec20
Key:22 Record:Rec22
Key:25 Record:Rec25
Key:30 Record:Rec30
Key:40 Record:Rec40
Key:45 Record:Rec45
Total matching records: 8

```

Explanation & Justification:

This program extends the Q5 B+ Tree design by attaching an actual data 'record' (value) to every key, so each leaf stores parallel keys[] and values[] lists — modelling how a real B+ Tree index maps a search key to an actual row of data.

15 records are inserted, causing the tree to grow through several leaf and internal-node splits as leaves fill beyond order-1 = 3 keys. The display_tree() function prints the resulting structure level by level, distinguishing 'internal' routing levels from the final 'leaf' level that holds all data.

inorder_traverse() walks the leftmost leaf and follows 'next' pointers across the entire leaf chain, producing every key in fully sorted order in a single linear pass — this is a direct consequence of B+ Tree leaf linking, and it is exactly how a database performs an 'ORDER BY primary_key' scan efficiently.

Justification: range_query(15, 45) locates the leaf containing key 15 directly (skipping the root-to-leaf traversal for every subsequent key) and then walks forward along the leaf chain, collecting every (key, record) pair until a key exceeding 45 is seen. The output confirms all keys in [15, 45] — namely 15, 17, 20, 22, 25, 30, 40, 45 — are returned correctly with their associated records, and the tree stops scanning as soon as it passes 45 rather than scanning the whole tree.